

Batata

- **Limite de tempo:** 10 minutos
- **Ambiente:** uma GPU (≈16 GB VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Armazenamento:** 5 GB

Tarefa

Seu amigo sugere jogar um jogo de adivinhação. Ele, como juiz, escolhe uma palavra oculta de um vocabulário fixo, e você deve encontrá-la em no máximo 30 turnos. A cada turno, o juiz compara duas palavras e informa qual delas é semanticamente mais próxima da palavra oculta. Toda partida começa com o par fixo `lamp` vs `potato`, porque essas são duas das coisas favoritas de seu amigo. Em seguida, seu programa propõe uma nova palavra. A vencedora da comparação é mantida e comparada com sua próxima proposta. Você vence uma partida no momento em que propõe exatamente a palavra oculta. A correspondência não diferencia maiúsculas de minúsculas. Toda palavra que você propuser deve estar em `dataset/vocabulary.json`.

Há um exemplo completo em `solution.ipynb` com o protocolo e o carregamento dos dados. Você pode alterar a classe `PublicEmbeddingPlayer`. Seu programa é inicializado uma vez e joga todas as partidas em uma única execução; o protocolo cria um novo `PublicEmbeddingPlayer` no início de cada partida.

O Juiz

Seu programa envia um objeto JSON ao Juiz, e o Juiz responde com um objeto JSON.

Um exemplo detalhado, com a palavra oculta mostrada apenas para explicar o protocolo:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Os turnos são indexados de 1 a 30.

As opções de `verdict` são `first`, significando que `word1` é mais próxima, `second`, significando que `word2` é mais próxima, ou `same`, significando que ambas as palavras são igualmente próximas da palavra oculta.

`winner_word` é a palavra mantida para a próxima comparação. Em um veredito `same`, a primeira palavra permanece.

Dataset

Compartilhados por todas as partições:

- `dataset/vocabulary.json` — 1602 palavras únicas em letras minúsculas. A palavra oculta é sempre uma delas.
- `dataset/public_embeddings.npy` — `float32`, formato (1602, 2560). A linha `i` corresponde à palavra `i` no vocabulário. Estes são *embeddings públicos*; o juiz usa uma representação diferente e privada.

As partições são conjuntos de palavras ocultas:

Partição	Palavras	Respostas	Use-a para
<code>test_public</code>	120	<code>dataset/test_public.json</code>	executar sua solução e calcular sua própria pontuação
<code>test_leaderboard_a</code>	120	ocultas	leaderboard ao vivo
<code>test_leaderboard_b</code>	120	ocultas	classificação final

Não há uma partição `train` — nada é ajustado a partir de linhas rotuladas.

Modelos fornecidos

Dois modelos de embedding pré-treinados são fornecidos com a tarefa e podem ser usados:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Ambos devem ser carregados a partir de seu caminho local; um ID do hub do Hugging Face, como "BAAI/bge-m3", inicia um download e falha, porque a avaliação é offline. Cada diretório contém um `example.py` executável que mostra a chamada offline.

Bibliotecas disponíveis: `numpy`, `torch`, `sentence-transformers`. Sem internet, sem downloads, sem outros pacotes.

Saída

Nenhuma. Esta é uma tarefa interativa: sua solução não grava um arquivo de resposta; ela se comunica com o juiz por `stdin/stdout`, conforme descrito acima.

Métrica

Uma partida resolvida no turno t recebe $1.0 - 0.02 \times \max(0, t - 10)$; uma partida não resolvida em até 30 turnos recebe 0. Portanto, os turnos 1-10 recebem 1.00, o turno 20 recebe 0.80, e o turno 30 recebe 0.60.

Sua pontuação na tarefa é a pontuação média das partidas $\times 100$, entre 0.00 e 100.00.

O limite de 10 minutos abrange a inicialização, a preparação e todas as 120 partidas no conjunto de teste.

Como enviar

1. Abra `solution.ipynb`, edite `PublicEmbeddingPlayer` e execute todas as células para garantir que esteja funcionando.
2. Opcionalmente, verifique-a localmente: `python local_test.py solution.ipynb --limit 5`. O juiz local usa os embeddings *públicos*, portanto sua pontuação é apenas uma referência.
3. Salve `solution.ipynb`.
4. Abra a aba Git na barra lateral esquerda do JupyterLab.
5. Faça stage de `solution.ipynb` (o ícone + ao lado dele).
6. Insira uma mensagem de commit e clique em Commit.
7. Clique no ícone de nuvem com uma seta para cima para fazer push.
8. Retorne a esta página da Competição e clique em Submit, para o commit com a mensagem desejada que foi fornecida.

Envie exatamente um arquivo, chamado `solution.ipynb`, abrangendo quaisquer preparações e inferências necessárias.